CSI2 Interface

for S32R41

Rev.0.8.10/16.12.2024

Chapter 1

Module Index

1.1 Software Modules

Here is a list of all modules:

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

Chapter 3

Module Documentation

3.1 CSI2

CSI2.

Modules

- CSI2 AUTOSAR Driver Constants

  CSI2 Driver Constants .
- CSI2 AUTOSAR Driver Data Types

  CSI2 Driver Data Types .
- CSI2 AUTOSAR Driver Functions

  CSI2 Driver Functions .

3.1.1 Detailed Description

CSI2.

Describes the functions, data types and structures that make up the CSI2 AUTOSAR Complex Device Driver user interface. Proper usage of the CSI2 Driver is described in CSI2 User Guide, in RSDK User Manual.

3.2 CSI2 AUTOSAR Driver Constants

CSI2 Driver Constants .

Macros

- #define CSI2_ERR_NO 0UL

  Errors definitions.
- #define CSI2_ERR_PHY_SYNC (1UL << 0u)
- #define CSI2_ERR_PHY_NO_SYNC (1UL << 1u)
- #define CSI2_ERR_PHY_ESC (1UL << 2u)
- #define CSI2_ERR_PHY_SESC (1UL << 3u)
- #define CSI2_ERR_PHY_CTRL (1UL << 4u)
- #define CSI2_ERR_PACK_ECC1 (1UL << 6u)
- #define CSI2_ERR_PACK_ECC2 (1UL << 7u)
- #define CSI2_ERR_PACK_SYNC (1UL << 8u)
- #define CSI2_ERR_PACK_DATA (1UL << 9u)
- #define CSI2_ERR_PACK_CRC (1UL << 10u)
- #define CSI2_ERR_PACK_ID (1UL << 11u)
- #define CSI2_ERR_LINE_LEN (1UL << 13u)
- #define CSI2_ERR_LINE_CNT (1UL << 14u)
- #define CSI2_ERR_LINE_LEN_MD (1UL << 15u)
- #define CSI2_ERR_LINE_CNT_MD (1UL << 16u)
- #define CSI2_ERR_HS_EXIT (1UL << 17u)
- #define CSI2_ERR_FIFO (1UL << 18u)
- #define CSI2_ERR_BUF_OVERFLOW (1UL << 19u)
- #define CSI2_ERR_AXI_OVERFLOW (1UL << 20u)
- #define CSI2_ERR_AXI_RESPONSE (1UL << 21u)
- #define CSI2_ERR_SPURIOUS_PHY (1UL << 22u)
- #define CSI2_ERR_SPURIOUS_PKT (1UL << 23u)
- #define CSI2_ERR_SPURIOUS_EVT (1UL << 24u)
- #define CSI2_EVT_FRAME_START (1UL << 0u)

  Events definitions.
- #define CSI2_EVT_FRAME_END (1UL << 1u)
- #define CSI2_EVT_SHORT_PACKET (1UL << 2u)
- #define CSI2_EVT_LINE_END (1UL << 3u)
- #define CSI2_EVT_BIT_NOT_TOGGLE (1UL << 5u)
- #define CSI2_EVT_NEXT_START_NOT_0 (1UL << 6u)
- #define CSI2_MIN_NR_LANES 1u
- #define CSI2_MIN_VC_BUF_NR_LINES 1u
- #define CSI2_MAX_ANTENNA_NR 4u
- #define CSI2_MIN_VAL_UNSIGNED 0x0000u
- #define CSI2_MIN_VAL_SIGNED 0xc000
- #define CSI2_MAX_VAL_UNSIGNED 0xffffu
- #define CSI2_MAX_VAL_SIGNED 0x7ff8
- #define CSI2_MAX_RX_FREQ 2500u
- #define CSI2_MIN_RX_FREQ 80u
- #define CSI2_OFFSET_AUTOCOMPUTE 0x7fff
- #define CSI2_LINE_STAT_LENGTH 80U
- #define CSI2_5TH_CHANNEL_ON 0x400u
- #define CSI2_NORM_DTYPE_MASK 0x3fu /* the mask for the normal data types to be received */
- #define CSI2_MAX_VC_MASK 0x3u /* the mask for max Virtual Channel */
- #define CSI2_CBUF0_ENA_MASK 0x100u /* the mask for first buffer */

3.2.1 Detailed Description

CSI2 Driver Constants .

3.2.2 Macro Definition Documentation

3.2.2.1 CSI2_5TH_CHANNEL_ON

```
#define CSI2_5TH_CHANNEL_ON 0x400u
```

bit to enable the fifth channel

3.2.2.2 CSI2_CBUF0_ENA_MASK

```
#define CSI2_CBUF0_ENA_MASK 0x100u /* the mask for first buffer */
```

3.2.2.3 CSI2_ERR_AXI_OVERFLOW

```
#define CSI2_ERR_AXI_OVERFLOW (1UL << 20u)
```

RDP error, AXI buffer overrun (BUFFOVFAXI)

3.2.2.4 CSI2_ERR_AXI_RESPONSE

```
#define CSI2_ERR_AXI_RESPONSE (1UL << 21u)
```

RDP error, AXI response error (ERRRESP)

3.2.2.5 CSI2_ERR_BUF_OVERFLOW

```
#define CSI2_ERR_BUF_OVERFLOW (1UL << 19u)
```

RDP error, internal buffer overrun (BUFFOVF)

3.2.2.6 CSI2_ERR_FIFO

```
#define CSI2_ERR_FIFO (1UL << 18u)
```

RDP error, internal FIFO error (FIFO_OVERFLOW_ERROR)

3.2.2.7 CSI2_ERR_HS_EXIT

```
#define CSI2_ERR_HS_EXIT (1UL << 17u)
```

RDP error, High Speed exit error (EXIT_HS_ERROR)

3.2.2.8 CSI2_ERR_LINE_CNT

```
#define CSI2_ERR_LINE_CNT (1UL << 14u)
```

Line count error.

3.2.2.9 CSI2_ERR_LINE_CNT_MD

```
#define CSI2_ERR_LINE_CNT_MD (1UL << 16u)
```

Line count error for metadata.

3.2.2.10 CSI2_ERR_LINE_LEN

```
#define CSI2_ERR_LINE_LEN (1UL << 13u)
```

Line length error.

3.2.2.11 CSI2_ERR_LINE_LEN_MD

```
#define CSI2_ERR_LINE_LEN_MD (1UL << 15u)
```

Line length error for metadata.

3.2.2.12 CSI2_ERR_NO

```
#define CSI2_ERR_NO 0UL
```

Errors definitions.

All defined errors for a unit, at all levels of the protocol. No errors

3.2.2.13 CSI2_ERR_PACK_CRC

```
#define CSI2_ERR_PACK_CRC (1UL << 10u)
```

Packet error - frame CRC error. The CRC expected value is reported in rsdkCsi2Report_t::expectedCRC, and the received one in rsdkCsi2Report_t::receivedCRC

3.2.2.14 CSI2_ERR_PACK_DATA

```
#define CSI2_ERR_PACK_DATA (1UL << 9u)
```

Packet error - frame data error.

3.2.2.15 CSI2_ERR_PACK_ECC1

```
#define CSI2_ERR_PACK_ECC1 (1UL << 6u)
```

Packet error - one bit error corrected, the error bit position in Csi2_ErrorReportType::eccOneBitErrPos.

3.2.2.16 CSI2_ERR_PACK_ECC2

```
#define CSI2_ERR_PACK_ECC2 (1UL << 7u)
```

Packet error - more than one bit error in packet, not corrected.

3.2.2.17 CSI2_ERR_PACK_ID

```
#define CSI2_ERR_PACK_ID (1UL << 11u)
```

Packet error - ID error, the received ID is reported in rsdkCsi2Report_t::invalidPacketID.

3.2.2.18 CSI2_ERR_PACK_SYNC

```
#define CSI2_ERR_PACK_SYNC (1UL << 8u)
```

Packet error - frame sync error.

3.2.2.19 CSI2_ERR_PHY_CTRL

```
#define CSI2_ERR_PHY_CTRL (1UL << 4u)
```

PHY error - illegal command sequence

3.2.2.20 CSI2_ERR_PHY_ESC

```
#define CSI2_ERR_PHY_ESC (1UL << 2u)
```

PHY error - escape synchronization not realized

3.2.2.21 CSI2_ERR_PHY_NO_SYNC

```
#define CSI2_ERR_PHY_NO_SYNC (1UL << 1u)
```

PHY error - synchronization not realized.

3.2.2.22 CSI2_ERR_PHY_SESC

```
#define CSI2_ERR_PHY_SESC (1UL << 3u)
```

PHY error - illegal command sequence in escape mode

3.2.2.23 CSI2_ERR_PHY_SYNC

```
#define CSI2_ERR_PHY_SYNC (1UL << 0u)
```

PHY error - one cycle synchronization pattern error (soft err)

3.2.2.24 CSI2_ERR_SPURIOUS_EVT

```
#define CSI2_ERR_SPURIOUS_EVT (1UL << 24u)
```

Spurious error at Data level.

3.2.2.25 CSI2_ERR_SPURIOUS_PHY

```
#define CSI2_ERR_SPURIOUS_PHY (1UL << 22u)
```

Spurious error at PHY level.

3.2.2.26 CSI2_ERR_SPURIOUS_PKT

```
#define CSI2_ERR_SPURIOUS_PKT (1UL << 23u)
```

Spurious error at Packet level.

3.2.2.27 CSI2_EVT_BIT_NOT_TOGGLE

```
#define CSI2_EVT_BIT_NOT_TOGGLE (1UL << 5u)
```

Bit not toggled on a channel, reported in rsdkCsi2Report_t::notToggledBits.

3.2.2.28 CSI2_EVT_FRAME_END

```
#define CSI2_EVT_FRAME_END (1UL << 1u)
```

Frame end event (FE)

3.2.2.29 CSI2_EVT_FRAME_START

```
#define CSI2_EVT_FRAME_START (1UL << 0u)
```

Events definitions.

All defined events for a Virtual Channel. Frame start event (FS)

3.2.2.30 CSI2_EVT_LINE_END

```
#define CSI2_EVT_LINE_END (1UL << 3u)
```

Line end event (LINEDONE)

3.2.2.31 CSI2_EVT_NEXT_START_NOT_0

```
#define CSI2_EVT_NEXT_START_NOT_0 (1UL << 6u)
```

Bit signaling that the next frame start will not be at the beginning of the buffer

3.2.2.32 CSI2_EVT_SHORT_PACKET

```
#define CSI2_EVT_SHORT_PACKET (1UL << 2u)
```

Generic short packet received (GNSP)

3.2.2.33 CSI2_LINE_STAT_LENGTH

```
#define CSI2_LINE_STAT_LENGTH 80U
```

the length of statistic data

3.2.2.34 CSI2_MAX_ANTENNA_NR

```
#define CSI2_MAX_ANTENNA_NR 4u
```

Maximum number of receiving channels (real/complex)

3.2.2.35 CSI2_MAX_RX_FREQ

```
#define CSI2_MAX_RX_FREQ 2500u
```

Max. 2.5Gbps input data for **S32R45**

3.2.2.36 CSI2_MAX_VAL_SIGNED

```
#define CSI2_MAX_VAL_SIGNED 0x7ff8
```

Definition for maximum signed sample value

3.2.2.37 CSI2_MAX_VAL_UNSIGNED

```
#define CSI2_MAX_VAL_UNSIGNED 0xffffu
```

Definition for maximum unsigned sample value

3.2.2.38 CSI2_MAX_VC_MASK

```
#define CSI2_MAX_VC_MASK 0x3u /* the mask for max Virtual Channel */
```

3.2.2.39 CSI2_MIN_NR_LANES

```
#define CSI2_MIN_NR_LANES 1u
```

Minimum number of physical lanes to be used

3.2.2.40 CSI2_MIN_RX_FREQ

```
#define CSI2_MIN_RX_FREQ 80u
```

Min. 80Mbps input data for **S32R45**

3.2.2.41 CSI2_MIN_VAL_SIGNED

```
#define CSI2_MIN_VAL_SIGNED 0xc000
```

Definition for minimum signed sample value

3.2.2.42 CSI2_MIN_VAL_UNSIGNED

```
#define CSI2_MIN_VAL_UNSIGNED 0x0000u
```

Definition for minimum unsigned sample value (default)

3.2.2.43 CSI2_MIN_VC_BUF_NR_LINES

```
#define CSI2_MIN_VC_BUF_NR_LINES 1u
```

Minimum number of VC to be defined

3.2.2.44 CSI2_NORM_DTYPE_MASK

```
#define CSI2_NORM_DTYPE_MASK 0x3fu /* the mask for the normal data types to be received */
```

3.2.2.45 CSI2_OFFSET_AUTOCOMPUTE

```
#define CSI2_OFFSET_AUTOCOMPUTE 0x7fff
```

Definition for auto computing offset for incoming data

3.3 CSI2 AUTOSAR Driver Data Types

CSI2 Driver Data Types .

Data Structures

- struct Csi2_ShortPacketType

  Structure for short packet received.
- struct Csi2_LineLenErrType

  Structure for line length error or number of lines received error in the received frame.
- struct Csi2_ChFrameStatType

  Structure for channel statistics, for the entire frame.
- struct Csi2_VCParamsType

  Structure for VirtualChannel configuration.
- struct Csi2_MetaDataParamsType

  Structure for VirtualChannel configuration of metadata.
- struct Csi2_ErrorReportType

  Structure which describe the errors/events signaled by an interrupt request.
- struct Csi2_SetupParamsType

  Structure for unit configuration.
- struct Csi2_VCDriverStateType

  Structure for VC driver parameters.
- struct Csi2_DriverParamsType

  Structure which keep the necessary parameters for run-time.

Typedefs

- typedef void(∗ Csi2_IsrCbType) (Csi2_ErrorReportType ∗pReportingStruct)

  Definition of callback function type to be called by the CSI2 interrupt handler.

Enumerations

- enum Csi2_DataStreamType {
  CSI2_DATA_TYPE_EMBD = 0x12u , CSI2_DATA_TYPE_YUV422_8 = 0x1Eu , CSI2_DATA_TYPE_YUV422_10 = 0x1Fu , CSI2_DATA_TYPE_RGB565 = 0x22u ,
  CSI2_DATA_TYPE_RGB888 = 0x24u , CSI2_DATA_TYPE_RAW8 = 0x2Au , CSI2_DATA_TYPE_RAW10 = 0x2Bu , CSI2_DATA_TYPE_RAW12 = 0x2Cu ,
  CSI2_DATA_TYPE_RAW14 = 0x2Du , CSI2_DATA_TYPE_RAW16 = 0x2Eu , CSI2_DATA_TYPE_USR0 = 0x30u , CSI2_DATA_TYPE_USR1 = 0x31u ,
  CSI2_DATA_TYPE_USR2 = 0x32u , CSI2_DATA_TYPE_USR3 = 0x33u , CSI2_DATA_TYPE_USR4 = 0x34u , CSI2_DATA_TYPE_USR5 = 0x35u ,
  CSI2_DATA_TYPE_16_FROM_8 = 0x36u , CSI2_DATA_TYPE_USR7 = 0x37u , CSI2_DATA_TYPE_AUX_0_NO_DROP = 0x40u , CSI2_DATA_TYPE_R12_A0_NO_DROP = 0x6Cu ,
  CSI2_DATA_TYPE_R16_A0_NO_DROP = 0x6Eu , CSI2_DATA_TYPE_AUX_0_DR_1OF2 = 0x80u , CSI2_DATA_TYPE_R12_A0_DR_1OF2 = 0xACu , CSI2_DATA_TYPE_R16_A0_DR_1OF2 = 0xAEu ,
  CSI2_DATA_TYPE_AUX_0_DR_3OF4 = 0xc0u , CSI2_DATA_TYPE_R12_A0_DR_3OF4 = 0xECu , CSI2_DATA_TYPE_R16_A0_DR_3OF4 = 0xEEu , CSI2_DATA_TYPE_AUX_1_NO_DROP = 0x100u ,
  CSI2_DATA_TYPE_R12_A1_NO_DROP = 0x12Cu , CSI2_DATA_TYPE_R16_A1_NO_DROP = 0x12Eu , CSI2_DATA_TYPE_MAX = 0x13Fu }

  The data types accepted by the MIPI-CSI2 interface.
- enum Csi2_UnitIdType { CSI2_UNIT_0 = 0 , CSI2_MAX_UNITS }

  CSI2 units enumeration.
- enum Csi2_VirtChnlIdType {
  CSI2_VC_0 = 0 , CSI2_VC_1 , CSI2_VC_2 , CSI2_VC_3 ,
  CSI2_MAX_VC }

  CSI2 Virtual Channel enumeration.

- enum Csi2_LaneIdType { CSI2_LANE_0 = 0 , CSI2_LANE_1 , CSI2_MAX_LANE }

  CSI2 lanes enumeration.
- enum Csi2_LaneStatusType { CSI2_LANE_STATE_MARK , CSI2_LANE_STATE_ULPA , CSI2_LANE_STATE_STOP , CSI2_LANE_STATE_REC , CSI2_LANE_STATE_VRX , CSI2_LANE_STATE_ON , CSI2_LANE_STATE_OFF , CSI2_LANE_STATE_ERR }

  CSI2 lanes status enumeration.
- enum Csi2_ChannelIdType { CSI2_CHANNEL_A = 0 , CSI2_CHANNEL_B , CSI2_CHANNEL_C , CSI2_CHANNEL_D , CSI2_MAX_CHANNEL }

  Enum for ADC channels.
- enum Csi2_AutoDCComputeTimeType { CSI2_AUTODC_NO = 0u , CSI2_AUTODC_EVERY_LINE = 1u , CSI2_AUTODC_AT_FE , CSI2_AUTODC_LAST_LINE , CSI2_AUTODC_NO_WITH_STATS , CSI2_AUTODC_MAX }

  Auto DC offset calculation (statistics management) type enum.
- enum Csi2_StatisticsType { CSI2_STAT_NO = 0u , CSI2_STAT_YES = 1u , CSI2_STAT_MAX }

  Auto DC offset calculation (statistics management) type enum.
- enum Csi2_DriverStateType { CSI2_DRIVER_STATE_NOT_INITIALIZED = 0 , CSI2_DRIVER_STATE_ON , CSI2_DRIVER_STATE_OFF , CSI2_DRIVER_STATE_STOP }

  Structure which keep the possible values for driver status.
- enum Csi2_DphySetupOptionsType { CSI2_DPHY_INIT_SIMPLE = 0u , CSI2_DPHY_INIT_SHORT_CALIB , CSI2_DPHY_INIT_W_STOP_STATE , CSI2_DPHY_INIT_SHORT_AND_STOP = CSI2_DPHY_INIT_SHORT_CALIB | CSI2_DPHY_INIT_W_↩ STOP_STATE , CSI2_DPHY_INIT_MAX = CSI2_DPHY_INIT_SHORT_AND_STOP + 1 }

  Setup options for DPHY layer.
- enum Csi2_BufferPointerResetType { CSI2_BUF_RESET_NO = 0u , CSI2_BUF_RESET_FS , CSI2_BUF_RESET_MAX }

  This type is defining the way the buffer pointer could be reset if some lines are rejected.
- enum Csi2_ADCManagementType { CSI2_ADC_NO = 0u , CSI2_ADC_H_ONLY , CSI2_ADC_F_ONLY , CSI2_ADC_HF , CSI2_ADC_MAX }

  Enum for ADC header/footer management.
- enum Csi2_DataManipulationType { CSI2_VC_BUF_REAL_DATA = 0x00U , CSI2_VC_BUF_COMPLEX_DATA = 0x10U , CSI2_VC_BUF_FIFTH_CH_OFF = 0x00U , CSI2_VC_BUF_FIFTH_CH_ON = 0x20U , CSI2_VC_BUF_5TH_CH_M0_NODROP = 0x00U , CSI2_VC_BUF_5TH_CH_M0_1_OF_2 = 0x40U , CSI2_VC_BUF_5TH_CH_M0_3_OF_4 = 0x80U , CSI2_VC_BUF_5TH_CH_MODE_1 = 0xc0U , CSI2_VC_BUF_OUT_ILEAVED = 0x00U , CSI2_VC_BUF_OUT_TILE8 = 0x100U , CSI2_VC_BUF_OUT_TILE16 = 0x200U , CSI2_VC_BUF_NO_FLIP_SIGN = 0x000U , CSI2_VC_BUF_FLIP_SIGN = 0x400U , CSI2_VC_BUF_SWAP_RAW8 = 0x800U , CSI2_VC_BUF_RAW16_MSB_F = 0x1000U , CSI2_VC_BUF_WRITE_ALL_DATA = 0x2000U }

  Input/output data definitions enumeration.
- enum Csi2_ExternalTriggerType { CSI2_REQ_ETRG_ENA_ON_ERROR = 0x01U , CSI2_REQ_ETRG_ENA_ON_PACKET = 0x02U , CSI2_REQ_ETRG_ENA_ON_PF = 0x04U }

  GPIO/SDMA trigger enablement enumeration.
- enum Csi2_ExternalEventsType { CSI2_REQ_ETRG_EVT_FRAME_START = 0x00U , CSI2_REQ_ETRG_EVT_FRAME_END = 0x10U , CSI2_REQ_ETRG_EVT_CHIRP_START = 0x20U , CSI2_REQ_ETRG_EVT_CHIRP_END = 0x30U , CSI2_REQ_ETRG_EVT_PACK_EMBD = 0x00U , CSI2_REQ_ETRG_EVT_PACK_USER = 0x04U , CSI2_REQ_ETRG_EVT_PACK_RAW = 0x08U , CSI2_REQ_ETRG_EVT_PACK_RGB = 0x0cu , CSI2_REQ_ETRG_EVT_ERR_LINECNT = 0x00U , CSI2_REQ_ETRG_EVT_ERR_LINLEN = 0x01U , CSI2_REQ_ETRG_EVT_ERR_CRCECC = 0x02U , CSI2_REQ_ETRG_EVT_ERR_NOSYNC = 0x03U }

GPIO/SDMA events definitions enumeration.

- enum Csi2_ApiFunctionIdType {
  CSI2_API_ID_SETUP = 0u , CSI2_API_ID_RX_STOP , CSI2_API_ID_RX_START , CSI2_API_ID_POWER_OFF ,
  CSI2_API_ID_POWER_ON , CSI2_API_ID_GET_INTERFACE , CSI2_API_ID_GET_LANE_STATUS ,
  CSI2_API_ID_GET_BUFFER_LEN ,
  CSI2_API_ID_GET_FRAMES , CSI2_API_ID_GET_CHANNEL_NUM , CSI2_API_ID_VERSION_INFO_CHECK
  }

  Function API enums for CSI2 driver.
- enum Csi2_DetailedErrorType {
  CSI2_E_DRV_WRG_UNIT_ID = 0u , CSI2_E_DRV_NULL_PARAM_PTR , CSI2_E_DRV_NULL_VC_PARAM_PTR , CSI2_E_DRV_VC_AUX_MISMATCH ,
  CSI2_E_DRV_NULL_ERR_CB_PTR , CSI2_E_DRV_NULL_ISR_CB_PTR , CSI2_E_DRV_INVALID_CHANNEL_NR , CSI2_E_DRV_INVALID_CALIB_MODE ,
  CSI2_E_DRV_INVALID_LANES_NR , CSI2_E_DRV_INVALID_CLOCK_FREQ , CSI2_E_DRV_INVALID_RX_SWAP , CSI2_E_DRV_INVALID_DATA_TYPE ,
  CSI2_E_DRV_INVALID_EVT_REQ , CSI2_E_DRV_INVALID_VC_PARAMS , CSI2_E_DRV_INVALID_DC_PARAMS , CSI2_E_DRV_INVALID_INIT_PARAMS ,
  CSI2_E_DRV_INVALID_BUF_RESET , CSI2_E_DRV_INVALID_ADC_STAT , CSI2_E_DRV_TOO_SMALL_BUFFER , CSI2_E_DRV_NO_SAMPLE_PER_CHIRP ,
  CSI2_E_DRV_NO_CHIRPS_PER_FRAME , CSI2_E_DRV_NO_LINE_LENGTH , CSI2_E_DRV_BUF_LEN_NOT_ALIGNED , CSI2_E_DRV_BUF_PTR_NULL ,
  CSI2_E_DRV_BUF_PTR_NOT_ALIGNED , CSI2_E_DRV_BUF_NUM_LINES_ERR , CSI2_E_DRV_ADC_STAT_FOR_EXTER , CSI2_E_DRV_ERR_UNIT_0_MUST_BE_FIRST ,
  CSI2_E_DRV_ERR_UNIT_1_MUST_BE_FIRST , CSI2_E_DRV_ERR_UNIT_2_MUST_BE_FIRST ,
  CSI2_E_DRV_ERR_INVALID_INT_NR , CSI2_E_DRV_ERR_INVALID_CORE_NR ,
  CSI2_E_DRV_ERR_IRQ_HANDLER_REG , CSI2_E_DRV_SW_RESET_ERROR , CSI2_E_DRV_CALIBRATION_TIMEOUT , CSI2_E_DRV_TIMER_ERROR ,
  CSI2_E_DRV_HW_RESPONSE_ERROR , CSI2_E_DRV_NOT_INIT , CSI2_E_DRV_ERR_INVALID_REQ ,
  CSI2_E_DRV_ERR_COPY_DATA_ERROR ,
  CSI2_E_DRV_ERR_EVT_CONN , CSI2_E_DRV_ERR_START_EVT_MGR , CSI2_E_DRV_WRG_STATE ,
  CSI2_E_DRV_POWERED_OFF ,
  CSI2_E_DRV_RX_STOPPED , CSI2_E_DRV_STATE_LINE_MARK , CSI2_E_DRV_STATE_LINE_ULPA ,
  CSI2_E_DRV_STATE_LINE_STOP ,
  CSI2_E_DRV_STATE_LINE_REC , CSI2_E_DRV_STATE_LINE_VRX , CSI2_E_DRV_STATE_LINE_ON ,
  CSI2_E_DRV_STATE_LINE_OFF ,
  CSI2_E_DRV_STATE_ON , CSI2_E_PARAM_POINTER_NULL }

  Detailed errors for AUTOSAR environment.
- enum Csi2_PinsSwapType {
  CSI2_PINS_NO_SWAP = 0x0u , CSI2_PINS_SWAP_CLOCK = 0x1u , CSI2_PINS_SWAP_LANE_0 = 0x2u , CSI2_PINS_SWAP_LANE_1 = 0x4u ,
  CSI2_PINS_SWAP_LANE_2 = 0x8u , CSI2_PINS_SWAP_LANE_3 = 0x10u }

  Enums for pins swap of the CSI2 PHY interface.

3.3.1 Detailed Description

CSI2 Driver Data Types .

3.3.2 Typedef Documentation

3.3.2.1 Csi2_IsrCbType

```
typedef void(* Csi2_IsrCbType) (Csi2_ErrorReportType *pReportingStruct)
```

Definition of callback function type to be called by the CSI2 interrupt handler.

This callback must be used for processing a CSI2 error/event interrupt.
Simple callback function definition example : void CallbackCsi2(rsdkCsi2Report_t *pStruct) { ... }

Parameters

in	*pReportingStruct**	- the callback receives a pointer to a complete description of the error/event

Returns

nothing

3.3.3 Enumeration Type Documentation

3.3.3.1 Csi2_ADCManagementType

```
enum Csi2_ADCManagementType
```

Enum for ADC header/footer management.

SAF85XX/SAF86XX offer, only for ADC input, the possibility to receive and record some header/footer data. The data is recorded at the end of each chirp data, according to the Statistics management.

Enumerator

CSI2_ADC_NO	No ADC Header/Footer data recorded. The buffer line length must not provide space for this.
CSI2_ADC_H_ONLY	No Statistics, only ADC data header recorder. The buffer line must be at least (chirp_data + 16 bytes).
CSI2_ADC_F_ONLY	No Statistics, only ADC data footer recorder. The buffer line must be at least (chirp_data + 16 bytes).
CSI2_ADC_HF	No Statistics, both ADC data header & footer recorder. The buffer line must be at least (chirp_data + 16 bytes).
CSI2_ADC_MAX	ADC data management max limit, to not be used in application

3.3.3.2 Csi2_ApiFunctionIdType

`enum Csi2_ApiFunctionIdType`

Function API enums for CSI2 driver.

Each API function has a specific ID, to make possible to detect the error calling environment.

Enumerator

CSI2_API_ID_SETUP	
CSI2_API_ID_RX_STOP	The ID for Csi2_Setup function
CSI2_API_ID_RX_START	The ID for Csi2_RxStop function
CSI2_API_ID_POWER_OFF	The ID for Csi2_RxStart function
CSI2_API_ID_POWER_ON	The ID for Csi2_PowerOff function
CSI2_API_ID_GET_INTERFACE	The ID for Csi2_PowerOn function
CSI2_API_ID_GET_LANE_STATUS	The ID for Csi2_GetInterfaceStatus function
CSI2_API_ID_GET_BUFFER_LEN	The ID for Csi2_GetLaneStatus function
CSI2_API_ID_GET_FRAMES	The ID for Csi2_GetBufferRealLineLen function
CSI2_API_ID_GET_CHANNEL_NUM	The ID for Csi2_GetFramesCounter function
CSI2_API_ID_VERSION_INFO_CHECK	The ID for Csi2_GetChannelNum function The ID for Csi2_GetVersionInfo function

3.3.3.3 Csi2_AutoDCComputeTimeType

`enum Csi2_AutoDCComputeTimeType`

Auto DC offset calculation (statistics management) type enum.

This enumeration is used for Csi2_SetupParamsType:statManagement and generally for the management of Statistics and AutoDcOffset. The general lines are :

- if CSI2_STATISTIC_DATA_USAGE is set to STD_OFF, no Statistics or AutoDC will be available, the field Csi2_SetupParamsType:statManagement will not be included to the setup structure.
- if CSI2_STATISTIC_DATA_USAGE is set to STD_ON, the Statistics and AutoDC will be available as described below. If the Statistics are required to be reported (any but not CSI2_AUTODC_NO), the line data length must be the necessary data amount plus a fixed length of 80 bytes for Statistics. If one of CSI2_AUTODC_EVERY_LINE, CSI2_AUTODC_AT_FE, CSI2_AUTODC_LAST_LINE is used but no one channel has CSI2_OFFSET_AUTOCOMPUTE specified for Csi2_VCParamsType:offsetCompReal, the AutoDC will not be activated (the specified values will be used for fixed DC compensation), but the Statistics will be added to the end of each chirp. If CSI2_AUTODC_NO or CSI2_AUTODC_NO_WITH_STATS are used, if CSI2_OFFSET_AUTOCOMPUTE is used for fixed DC offset, an error will be raised.

Enumerator

CSI2_AUTODC_NO	Nor Statistics or auto DC compute required, the necessary buffer line length is the data length
CSI2_AUTODC_EVERY_LINE	Auto compute DC management after every line received
CSI2_AUTODC_AT_FE	Auto compute DC management only at Frame End. All statistics in the receiving buffer (according to bufNumLines for the Virtual Channel) will be processed
CSI2_AUTODC_LAST_LINE	AutoDC computed using only the chirp received in the first buffer line will be processed.
CSI2_AUTODC_NO_WITH_STATS	AutoDC not computed but Statistics will be reported at the end of the chirp.
CSI2_AUTODC_MAX	Statistics max limit, to not be used in application

3.3.3.4 Csi2_BufferPointerResetType

`enum Csi2_BufferPointerResetType`

This type is defining the way the buffer pointer could be reset if some lines are rejected.

For SAF85XX/SAF86XX is possible to specify how the buffer pointer is reset in case of data error. If no lines are rejected, this value is not relevant, only if at least one line is rejected.

Enumerator

CSI2_BUF_RESET_NO	Buffer pointer is not reset; if a line is rejected, the data is written in buffer using the next buffer line. For correct management of the data, application must use one of RsdkCsi2GetFirstByteOffset or RsdkCsi2GetFirstLinePos functions to detect where the next frame "start".
CSI2_BUF_RESET_FS	Buffer pointer is reset to 0 after Frame Start, so each frame will start at line 0 in the defined buffer; no other checks are necessary
CSI2_BUF_RESET_MAX	Buffer reset max limit, to not be used in application

3.3.3.5 Csi2_ChannelIdType

`enum Csi2_ChannelIdType`

Enum for ADC channels.

Channels E/F/G and H are available only for complex data input from the Front End (see CSI2_VC_BUF_REAL_DATA / CSI2_VC_BUF_COMPLEX_DATA). For real data only channels A/B/C and D are available, according to the settings.
Do not use **CSI2_MAX_CHANNEL** in procedures calls.

Enumerator

CSI2_CHANNEL_A	
CSI2_CHANNEL_B	
CSI2_CHANNEL_C	
CSI2_CHANNEL_D	
CSI2_MAX_CHANNEL	

3.3.3.6 Csi2_DataManipulationType

`enum Csi2_DataManipulationType`

Input/output data definitions enumeration.

All defined parameters for data flow manipulation.

Enumerator

CSI2_VC_BUF_REAL_DATA	The data coming is real (default)
CSI2_VC_BUF_COMPLEX_DATA	The data coming is complex
CSI2_VC_BUF_FIFTH_CH_OFF	Disable the fifth channel (default)
CSI2_VC_BUF_FIFTH_CH_ON	Enable the fifth channel
CSI2_VC_BUF_5TH_CH_M0_NODROP	The data coming from fifth channel not dropped (default)
CSI2_VC_BUF_5TH_CH_M0_1_OF_2	The data coming from fifth channel dropped one of two
CSI2_VC_BUF_5TH_CH_M0_3_OF_4	The data coming from fifth channel dropped three of four
CSI2_VC_BUF_5TH_CH_MODE_1	The data coming from fifth channel not dropped (mode one)
CSI2_VC_BUF_OUT_ILEAVED	The data is output interleaved (default)
CSI2_VC_BUF_OUT_TILE8	The data is output tile8
CSI2_VC_BUF_OUT_TILE16	The data is output tile16
CSI2_VC_BUF_NO_FLIP_SIGN	The data is output as unsigned (default)
CSI2_VC_BUF_FLIP_SIGN	The data sign is fliped
CSI2_VC_BUF_SWAP_RAW8	Swap bytes for RAW8 data received

Enumerator

CSI2_VC_BUF_RAW16_MSB_F	For RAW16, first is the MSB (not the LSB, as default)
CSI2_VC_BUF_WRITE_ALL_DATA	Received data before FrameStart, but after setup, is written into the memory

3.3.3.7 Csi2_DataStreamType

`enum Csi2_DataStreamType`

The data types accepted by the MIPI-CSI2 interface.

The data types accepted by the MIPI-CSI2 interface.

Enumerator

CSI2_DATA_TYPE_EMBD	type embedded
CSI2_DATA_TYPE_YUV422_8	YUV422 - 8 bits
CSI2_DATA_TYPE_YUV422_10	YUV422 - 10 bits
CSI2_DATA_TYPE_RGB565	RGB565
CSI2_DATA_TYPE_RGB888	RGB888
CSI2_DATA_TYPE_RAW8	RAW-8
CSI2_DATA_TYPE_RAW10	RAW-10
CSI2_DATA_TYPE_RAW12	RAW-12 - radar default
CSI2_DATA_TYPE_RAW14	RAW-14
CSI2_DATA_TYPE_RAW16	RAW-14
CSI2_DATA_TYPE_USR0	user defined data ...
CSI2_DATA_TYPE_USR1	
CSI2_DATA_TYPE_USR2	
CSI2_DATA_TYPE_USR3	
CSI2_DATA_TYPE_USR4	
CSI2_DATA_TYPE_USR5	
CSI2_DATA_TYPE_16_FROM_8	specific data type; data will be received as RAW8 data, but the output will be as 16 bits data, one sample will have 2 bytes of data
CSI2_DATA_TYPE_USR7	... user defined data
CSI2_DATA_TYPE_AUX_0_NO_DROP	auxiliary, mode 0, no drop, mask
CSI2_DATA_TYPE_R12_A0_NO_DROP	RAW12 + auxiliary, mode 0, no drop, type to be used

Enumerator

CSI2_DATA_TYPE_R16_A0_NO_DROP	RAW16 + auxiliary, mode 0, no drop, type to be used
CSI2_DATA_TYPE_AUX_0_DR_1OF2	auxiliary, mode 0, drop 1 of 2, mask
CSI2_DATA_TYPE_R12_A0_DR_1OF2	RAW12 + auxiliary, mode 0, drop 1 of 2, type to be used
CSI2_DATA_TYPE_R16_A0_DR_1OF2	RAW16 + auxiliary, mode 0, drop 1 of 2, type to be used
CSI2_DATA_TYPE_AUX_0_DR_3OF4	auxiliary, mode 0, drop 3 of 4, mask
CSI2_DATA_TYPE_R12_A0_DR_3OF4	RAW12 + auxiliary, mode 0, drop 3 of 4, type to be used
CSI2_DATA_TYPE_R16_A0_DR_3OF4	RAW16 + auxiliary, mode 0, drop 3 of 4, type to be used
CSI2_DATA_TYPE_AUX_1_NO_DROP	auxiliary, mode 1, no drop, mask
CSI2_DATA_TYPE_R12_A1_NO_DROP	RAW12 + auxiliary, mode 1, no drop, type to be used
CSI2_DATA_TYPE_R16_A1_NO_DROP	RAW16 + auxiliary, mode 1, no drop, type to be used
CSI2_DATA_TYPE_MAX	stream type maximum, not used

3.3.3.8 Csi2_DetailedErrorType

`enum Csi2_DetailedErrorType`

Detailed errors for AUTOSAR environment.

These errors are in synch with the general RSDK errors defined in the rsdkStatus.h file.

Enumerator

CSI2_E_DRV_WRG_UNIT_ID	Wrong unit ID specified
CSI2_E_DRV_NULL_PARAM_PTR	Wrong parameters pointer (NULL)
CSI2_E_DRV_NULL_VC_PARAM_PTR	Wrong parameters pointer to VC (NULL)
CSI2_E_DRV_VC_AUX_MISMATCH	Mismatch between VC and Auxiliary pointers (VC==NULL, Aux!=NULL)
CSI2_E_DRV_NULL_ERR_CB_PTR	Wrong error callback pointer (NULL)
CSI2_E_DRV_NULL_ISR_CB_PTR	Wrong error callback pointer for isr register (NULL)
CSI2_E_DRV_INVALID_CHANNEL_NR	Wrong channel/antenna number
CSI2_E_DRV_INVALID_CALIB_MODE	Wrong calibration ID
CSI2_E_DRV_INVALID_LANES_NR	Wrong lanes number
CSI2_E_DRV_INVALID_CLOCK_FREQ	Clock frequency incorrect (null)
CSI2_E_DRV_INVALID_RX_SWAP	Invalid Rx lanes swap specification
CSI2_E_DRV_INVALID_DATA_TYPE	Invalid data type specification or mismatch for auxiliary data
CSI2_E_DRV_INVALID_EVT_REQ	Wrong line end request : bufNumLinesTrigger is 0

Enumerator

CSI2_E_DRV_INVALID_VC_PARAMS	Invalid data type specification
CSI2_E_DRV_INVALID_DC_PARAMS	Invalid DC params compensation specification
CSI2_E_DRV_INVALID_INIT_PARAMS	Invalid params for DPHY initialization
CSI2_E_DRV_INVALID_BUF_RESET	Invalid request for buffer pointer reset
CSI2_E_DRV_INVALID_ADC_STAT	Invalid request for data header/footer management.
CSI2_E_DRV_TOO_SMALL_BUFFER	The buffer provided is too small for the requested data
CSI2_E_DRV_NO_SAMPLE_PER_CHIRP	Samples per chirp incorrect (null)
CSI2_E_DRV_NO_CHIRPS_PER_FRAME	Chirps per frame incorrect (null)
CSI2_E_DRV_NO_LINE_LENGTH	The VC buffer length was not provided
CSI2_E_DRV_BUF_LEN_NOT_ALIGNED	The VC buffer length is not aligned to 16 bytes boundaries
CSI2_E_DRV_BUF_PTR_NULL	The VC buffer pointer not provided
CSI2_E_DRV_BUF_PTR_NOT_ALIGNED	The VC buffer pointer not aligned to 16 bytes boundaries
CSI2_E_DRV_BUF_NUM_LINES_ERR	The VC buffer must have at least 2 lines
CSI2_E_DRV_ADC_STAT_FOR_EXTERN	Incorrect request for ADC data header/footer request and external source
CSI2_E_DRV_ERR_UNIT_0_MUST_BE_FIRST	Error : initialize unit_0 before unit_1
CSI2_E_DRV_ERR_UNIT_1_MUST_BE_FIRST	Error : initialize unit_1 before unit_0
CSI2_E_DRV_ERR_UNIT_2_MUST_BE_FIRST	Error : initialize unit_2 before unit_3
CSI2_E_DRV_ERR_INVALID_INT_NR	Incorrect IRQ nr used
CSI2_E_DRV_ERR_INVALID_CORE_NR	Incorrect core nr used
CSI2_E_DRV_ERR_IRQ_HANDLER_REG	Irq handler registration error
CSI2_E_DRV_SW_RESET_ERROR	Hardware error to reset sequence.
CSI2_E_DRV_CALIBRATION_TIMEOUT	Autocalibration error.
CSI2_E_DRV_TIMER_ERROR	Timer error waiting for STOP state on data lanes at initialization
CSI2_E_DRV_HW_RESPONSE_ERROR	Incorrect hardware response
CSI2_E_DRV_NOT_INIT	CSI2 driver not initialized
CSI2_E_DRV_ERR_INVALID_REQ	Incorrect request to the driver - Linux usage only
CSI2_E_DRV_ERR_COPY_DATA_ERROR	The data couldn't be copied between kernel<->user_space - Linux only
CSI2_E_DRV_ERR_EVT_CONN	Error when trying to connect to kernel events
CSI2_E_DRV_ERR_START_EVT_MGR	Error starting the events manager
CSI2_E_DRV_WRG_STATE	The interface is in a wrong state and the action can't succeed.
CSI2_E_DRV_POWERED_OFF	The PHY interface if powered off.
CSI2_E_DRV_RX_STOPPED	The Rx interface if stopped.
CSI2_E_DRV_STATE_LINE_MARK	The Rx lane in Mark-1 state.
CSI2_E_DRV_STATE_LINE_ULPA	The Rx lane in Ultra Low Power State.
CSI2_E_DRV_STATE_LINE_STOP	The Rx lane in Stop state.
CSI2_E_DRV_STATE_LINE_REC	The Rx lane is receiving data.
CSI2_E_DRV_STATE_LINE_VRX	The Rx lane is receiving valid data.
CSI2_E_DRV_STATE_LINE_ON	The Rx lane active, but not currently receiving data.
CSI2_E_DRV_STATE_LINE_OFF	The Rx lane inactive.
CSI2_E_DRV_STATE_ON	The driver status is ON.
CSI2_E_PARAM_POINTER_NULL	The provided pointer parameter is NULL

3.3.3.9 Csi2_DphySetupOptionsType

`enum Csi2_DphySetupOptionsType`

Setup options for DPHY layer.

There are few options for DPHY setup which application can ask. The two main options are : calibration speed-up and STOP state on data lanes at setup time

Enumerator

CSI2_DPHY_INIT_SIMPLE	Normal calibration , no STOP state to be reached
CSI2_DPHY_INIT_SHORT_CALIB	Short calibration, with previous calibration values, no STOP state to be reached
CSI2_DPHY_INIT_W_STOP_STATE	Normal calibration, STOP state to be reached in about 1.2ms or setup error reported
CSI2_DPHY_INIT_SHORT_AND_STOP	Short calibration, STOP state to be reached in about 1.2ms
CSI2_DPHY_INIT_MAX	Setup max limit, not to be used in application

3.3.3.10 Csi2_DriverStateType

`enum Csi2_DriverStateType`

Structure which keep the possible values for driver status.

Enumerator

CSI2_DRIVER_STATE_NOT_INITIALIZED	
CSI2_DRIVER_STATE_ON	
CSI2_DRIVER_STATE_OFF	
CSI2_DRIVER_STATE_STOP	

3.3.3.11 Csi2_ExternalEventsType

`enum Csi2_ExternalEventsType`

GPIO/SDMA events definitions enumeration.

All defined GPIO/SDMA events to be handled, for all defined triggers.

Enumerator

CSI2_REQ_ETRG_EVT_FRAME_START	Default, frame start will do a GPIO trigger
CSI2_REQ_ETRG_EVT_FRAME_END	Trigger at frame end
CSI2_REQ_ETRG_EVT_CHIRP_START	Trigger at chirp start
CSI2_REQ_ETRG_EVT_CHIRP_END	Trigger at chirp end
CSI2_REQ_ETRG_EVT_PACK_EMBD	Default, trigger on embedded data reception start
CSI2_REQ_ETRG_EVT_PACK_USER	Trigger on user data packet
CSI2_REQ_ETRG_EVT_PACK_RAW	Trigger on RAW packet reception
CSI2_REQ_ETRG_EVT_PACK_RGB	Trigger on RGB data packet
CSI2_REQ_ETRG_EVT_ERR_LINECNT	Default, trigger on line count error
CSI2_REQ_ETRG_EVT_ERR_LINLEN	Trigger on line length error
CSI2_REQ_ETRG_EVT_ERR_CRCECC	Trigger on CRC or ECC error
CSI2_REQ_ETRG_EVT_ERR_NOSYNC	Trigger on synchronization missing

3.3.3.12 Csi2_ExternalTriggerType

`enum Csi2_ExternalTriggerType`

GPIO/SDMA trigger enablement enumeration.

All defined GPIO/SDMA triggers to be handled.

Enumerator

CSI2_REQ_ETRG_ENA_ON_ERROR	Enable external trigger on VC error (linecount, line len, crc/ecc, no sync)
CSI2_REQ_ETRG_ENA_ON_PACKET	Enable external trigger on VC packet received
CSI2_REQ_ETRG_ENA_ON_PF	Enable external trigger on VC packet and frame

3.3.3.13 Csi2_LaneIdType

`enum Csi2_LaneIdType`

CSI2 lanes enumeration.

Depending the usage, this can means the lane position or number of lines used

Enumerator

CSI2_LANE_0	first lane / one lane
CSI2_LANE_1	second lane / two lanes
CSI2_MAX_LANE	lanes (maximum) per CSI2 unit, to not use in procedure call

3.3.3.14 Csi2_LaneStatusType

`enum Csi2_LaneStatusType`

CSI2 lanes status enumeration.

Depending the interface moment, a lane can be in one of these states.

Enumerator

CSI2_LANE_STATE_MARK	The Rx lane in Mark-1 state.
CSI2_LANE_STATE_ULPA	The Rx lane in Ultra Low Power State.
CSI2_LANE_STATE_STOP	The Rx lane in Stop state.
CSI2_LANE_STATE_REC	The Rx lane is receiving data.
CSI2_LANE_STATE_VRX	The Rx lane is receiving valid data.
CSI2_LANE_STATE_ON	The Rx lane active, but not currently receiving data.
CSI2_LANE_STATE_OFF	The Rx lane inactive.
CSI2_LANE_STATE_ERR	The requested parameters are not correct.

3.3.3.15 Csi2_PinsSwapType

`enum Csi2_PinsSwapType`

Enums for pins swap of the CSI2 PHY interface.

These masks must be used only if is necessary to change the receiving polarity of the pins, as the hardware design require. If the swap in not necessary, no data will be received on the PHY interface.

Enumerator

CSI2_PINS_NO_SWAP	No swap at all
CSI2_PINS_SWAP_CLOCK	Swap the clock polarity
CSI2_PINS_SWAP_LANE↩_0	Swap the lane0 polarity
CSI2_PINS_SWAP_LANE↩_1	Swap the lane1 polarity
CSI2_PINS_SWAP_LANE↩_2	Swap the lane2 polarity - not to be used for SAF85XX/SAF86XX
CSI2_PINS_SWAP_LANE↩_3	Swap the lane3 polarity - not to be used for SAF85XX/SAF86XX

3.3.3.16 Csi2_StatisticsType

`enum Csi2_StatisticsType`

Auto DC offset calculation (statistics management) type enum.

If auto computation for DC offset is required, only one of these value is accepted. If no auto computation for DC offset required, these values are ignored.

Enumerator

CSI2_STAT_NO	Statistics not required to be added by the interface
CSI2_STAT_YES	Statistics required to be added after the received data
CSI2_STAT_MAX	Statistics max limit, to not be used in application

3.3.3.17 Csi2_UnitIdType

`enum Csi2_UnitIdType`

CSI2 units enumeration.

Adjusted to the specific platform.

Enumerator

CSI2_UNIT_0	First unit (MIPICSI2_0)
CSI2_MAX_UNITS	The units limit, to not use in procedure call

3.3.3.18 Csi2_VirtChnlIdType

enum Csi2_VirtChnlIdType

CSI2 Virtual Channel enumeration.

Adjusted to the specific platform.

Enumerator

CSI2_VC_0	Virtual Channel 0
CSI2_VC_1	Virtual Channel 1
CSI2_VC_2	Virtual Channel 2
CSI2_VC_3	Virtual Channel 3
CSI2_MAX_VC	Virtual channels number limit for Csi2 unit, to not use in procedure call

3.4 CSI2 AUTOSAR Driver Functions

CSI2 Driver Functions .

Functions

- Std_ReturnType Csi2_Setup (const Csi2_UnitIdType unitId, const Csi2_SetupParamsType *setupParamPtr)
  This function prepare the CSI2 interface to receive data.
- Std_ReturnType Csi2_RxStop (const Csi2_UnitIdType unitId)
  The function stop the CSI2 receive module.
- Std_ReturnType Csi2_RxStart (const Csi2_UnitIdType unitId)
  The function start the CSI2 receive module.
- Std_ReturnType Csi2_PowerOff (const Csi2_UnitIdType unitId)
  The function power off the CSI2 module.
- Std_ReturnType Csi2_PowerOn (const Csi2_UnitIdType unitId)
  The function power on the CSI2 module.
- Csi2_LaneStatusType Csi2_GetInterfaceStatus (const Csi2_UnitIdType unitId)
  The function ask for the current status of the interface.
- Csi2_LaneStatusType Csi2_GetLaneStatus (const Csi2_UnitIdType unitId, const Csi2_LaneIdType laneId)
  The function ask for the current status of the specific lane.
- uint32 Csi2_GetBufferRealLineLen (const Csi2_DataStreamType dataType, const uint32 numChannels, const uint32 samplesPerChirp, const Csi2_StatisticsType autoStatistics)
  Procedure to get the real buffer length for a line/chirp.
- uint32 Csi2_GetFramesCounter (const Csi2_UnitIdType unitId, const Csi2_VirtChnlIdType vcId)
  The function return the current frames counter.
- uint8 Csi2_GetChannelNum (const Csi2_VCDriverStateType *pVCState)
  Get the real channels number of the VC..

3.4.1 Detailed Description

CSI2 Driver Functions .

3.4.2 Function Documentation

3.4.2.1 Csi2_GetBufferRealLineLen()

```
uint32 Csi2_GetBufferRealLineLen (
            const Csi2_DataStreamType dataType,
            const uint32 numChannels,
            const uint32 samplesPerChirp,
            const Csi2_StatisticsType autoStatistics )
```

Procedure to get the real buffer length for a line/chirp.

The procedure compute the necessary buffer length according to the input parameters.

Note

Because the length include the line/chirp statistics, autoStatistics must be 1 even only one channel in the entire Virtual Channel need the autoStatistics computation.

Parameters

in	*dataType*	Type of data to receive, normally RSDK_CSI2_DATA_TYPE_RAW12 for radar
in	*numChannels*	Number of channels, 1 ... 8; usually this number is the number of active antennas, multiply by 2 if complex (Re & Im) acquisition
in	*samplesPerChirp*	Samples per chirp and channel
in	*autoStatistics*	Value to inform that statistics must be added at the end of the chirp

Returns

The necessary memory amount for one line (chirp) if not 0.
Wrong input parameter if 0.

3.4.2.2 Csi2_GetChannelNum()

```
uint8 Csi2_GetChannelNum (
            const Csi2_VCDriverStateType * pVCState )
```

Get the real channels number of the VC..

Parameters

in	*pVCState*	- pointer to VC driver state

Returns

The channels number

3.4.2.3 Csi2_GetFramesCounter()

```
uint32 Csi2_GetFramesCounter (
            const Csi2_UnitIdType unitId,
            const Csi2_VirtChnlIdType vcId )
```

The function return the current frames counter.

The application can read the counter of frames, to decide how to manage the data. The counter is increased after handling the Frame End interrupt request.
A correct unitId and vcId must be provided.

Parameters

in	*unitId*	- unit identifier
in	*vcId*	- Virtual Channe identifier

Returns

uint32 - the current frames counter, 0xffffffff is not a valid value

3.4.2.4 Csi2_GetInterfaceStatus()

```
Csi2_LaneStatusType Csi2_GetInterfaceStatus (
            const Csi2_UnitIdType unitId )
```

The function ask for the current status of the interface.

The function return the status of the interface at the moment of the call.

Parameters

in	*unitId*	- unit identifier

Returns

Csi2_LaneStatusType - the current status of lane 0

Precondition

It can be called at any time, after a successful setup, to check the current status of the interface. Depending of the returned status, some times is necessary to loop on this call to really understand how interface is working.

3.4.2.5 Csi2_GetLaneStatus()

```
Csi2_LaneStatusType Csi2_GetLaneStatus (
            const Csi2_UnitIdType unitId,
            const Csi2_LaneIdType laneId )
```

The function ask for the current status of the specific lane.

The function return the status of the interface or specific lane at the moment of the call.

Parameters

in	*unitId*	- unit identifier
in	*laneId*	- lane identifier

Returns

Csi2_LaneStatusType - the current status of lane 0

Precondition

It can be called at any time, to check the current status of the interface/lane. Depending of the returned status, some times is necessary to loop on this call to really understand how lane is working.

3.4.2.6 Csi2_PowerOff()

```
Std_ReturnType Csi2_PowerOff (
            const Csi2_UnitIdType unitId )
```

The function power off the CSI2 module.

The interface is powered off.

Parameters

in	*unitId*	- unit identifier

Returns

Std_ReturnType - success or error, detailed status information reported on Det.

Precondition

It can be called when the reception need to be stopped and/or the interface to be powered off. The unit must be in INITIALIZED state.

3.4.2.7 Csi2_PowerOn()

```
Std_ReturnType Csi2_PowerOn (
            const Csi2_UnitIdType unitId )
```

The function power on the CSI2 module.

The interface is powered on. This action must be done only after a Csi2PowerOff was done before. After this call, the last setup parameters will be used to reinitialize the interface.

Parameters

in	*unitId*	- unit identifier

Returns

Std_ReturnType - success or error, detailed status information reported on Det.

Precondition

It can be called when the unit is powered off, or a restart is intended. If the Csi2_Setup call was not done before, an error will be reported, else the unit will be resetup using the previous setup parameters.

3.4.2.8 Csi2_RxStart()

```
Std_ReturnType Csi2_RxStart (
            const Csi2_UnitIdType unitId )
```

The function start the CSI2 receive module.

The receive module of the CSI2 interface is started. This action must be done if a CSI2_RxStop was done before, and the reception must be done at the same parameters.

Parameters

in	*unitId*	- unit identifier

Returns

Std_ReturnType - success or error, detailed status information reported on Det.

Precondition

It can be called when the reception is stopped, after an previous Csi2_RxStop.

3.4.2.9 Csi2_RxStop()

```
Std_ReturnType Csi2_RxStop (
            const Csi2_UnitIdType unitId )
```

The function stop the CSI2 receive module.

The receive module of the CSI2 interface is stopped. The receive will stop only after the end of the currently received packet.

Parameters

in	*unitId*	- unit identifier

Returns

Std_ReturnType - success or error, detailed status information reported on Det.

Precondition

It can be called when the reception need to be stopped. The unit must be in INITIALIZED state.

3.4.2.10 Csi2_Setup()

```
Std_ReturnType Csi2_Setup (
            const Csi2_UnitIdType unitId,
            const Csi2_SetupParamsType * setupParamPtr )
```

This function prepare the CSI2 interface to receive data.

The detailed actions done by this function :

- verify the input parameters; if any wrong parameter, error is returned
- reset the interface
- setup the CSI2 interface
- setup the interrupt vectors
- start the interface
- reset the frame counters The function can be called at any time, if necessary. If the result is not E_OK, the interface status is NOT_INITIALIZED, no matter was the status at call time.

Parameters

in	*unitId*	- the unit id to be used
in	*setupParamPtr*	- pointer to the CSI2 Driver setup structure

Returns

Std_ReturnType - success or error, detailed status information reported on Det.

Precondition

It must be called in the following situations:

- at application start (e.g. after boot)
- whenever the system parameters described in setupParamPtr need to be changed
- every time the Csi2_GetInterfaceStatus returns an error

Chapter 4

Data Structure Documentation

4.1 Csi2_ChFrameStatType Struct Reference

Structure for channel statistics, for the entire frame.

`#include <Csi2_Types.h>`

Data Fields

- sint64 channelSum
- sint16 channelDC
- sint16 reqChannelDC
- sint16 channelMin
- sint16 channelMax
- uint16 channelBitToggle

4.1.1 Detailed Description

Structure for channel statistics, for the entire frame.

This is for driver internal purposes at frame level.

4.1.2 Field Documentation

4.1.2.1 channelBitToggle

`uint16 channelBitToggle`

4.1.2.2 channelDC

```
sint16 channelDC
```

4.1.2.3 channelMax

```
sint16 channelMax
```

4.1.2.4 channelMin

```
sint16 channelMin
```

4.1.2.5 channelSum

```
sint64 channelSum
```

4.1.2.6 reqChannelDC

```
sint16 reqChannelDC
```

4.2 Csi2_DriverParamsType Struct Reference

Structure which keep the necessary parameters for run-time.

```
#include <Csi2_Types.h>
```

Data Fields

- Csi2_DriverStateType driverState
- Csi2_AutoDCComputeTimeType statisticsFlag
- Csi2_IsrCbType pCallback [MIPICSI2_1_INT2_IRQn - MIPICSI2_1_INT0_IRQn+1u]
- Csi2_VCDriverStateType workingParamVC [CSI2_MAX_VC]

4.2.1 Detailed Description

Structure which keep the necessary parameters for run-time.

4.2.2 Field Documentation

4.2.2.1 driverState

```
Csi2_DriverStateType driverState
```

4.2.2.2 pCallback

```
Csi2_IsrCbType pCallback[MIPICSI2_1_INT2_IRQn - MIPICSI2_1_INT0_IRQn+1u]
```

4.2.2.3 statisticsFlag

```
Csi2_AutoDCComputeTimeType statisticsFlag
```

4.2.2.4 workingParamVC

```
Csi2_VCDriverStateType workingParamVC[CSI2_MAX_VC]
```

4.3 Csi2_ErrorReportType Struct Reference

Structure which describe the errors/events signaled by an interrupt request.

```
#include <Csi2_Types.h>
```

Data Fields

- uint8 unitId
- uint32 errMaskU
- uint32 errMaskVC [CSI2_MAX_VC]
- uint8 evtMaskVC [CSI2_MAX_VC]
- uint8 invalidPacketID [CSI2_MAX_VC]
- uint8 eccOneBitErrPos [CSI2_MAX_VC]
- uint16 expectedCRC [CSI2_MAX_VC]
- uint16 receivedCRC [CSI2_MAX_VC]
- uint16 notToggledBits [CSI2_MAX_CHANNEL]
- uint8 lastDroppedData [CSI2_MAX_VC]
- Csi2_LineLenErrType lineLengthErr [CSI2_MAX_VC]
- Csi2_ShortPacketType shortPackets [CSI2_MAX_VC]

4.3.1 Detailed Description

Structure which describe the errors/events signaled by an interrupt request.

The cumulated errors/events are put in this structure. Normally only the configured Virtual Channels will be reported.

4.3.2 Field Documentation

4.3.2.1 eccOneBitErrPos

```
uint8 eccOneBitErrPos[CSI2_MAX_VC]
```

The recovered position for ECC one bit error, if CSI2_ERR_PACK_ECC1 reported

4.3.2.2 errMaskU

```
uint32 errMaskU
```

The cumulated masks for signaled errors at unit level; the possible masks are for:

- PHY level errors, i.e. CSI2_ERR_PHY_SYNC
- packet errors, i.e. CSI2_ERR_PACK_ECC1

4.3.2.3 errMaskVC

```
uint32 errMaskVC[CSI2_MAX_VC]
```

The cumulated masks for signaled errors at VC level; the possible masks are for:

- line errors definitions, i.e. CSI2_ERR_LINE_LEN
- all other miscellaneous

4.3.2.4 evtMaskVC

```
uint8 evtMaskVC[CSI2_MAX_VC]
```

Events mask for VCs, according to the requested events as required at setup - Csi2_VCParamsType->vcEventsReq the possible masks are defined by:

- events masks : CSI2_EVT_FRAME_START CSI2_EVT_FRAME_END CSI2_EVT_LINE_END CSI2_EVT_SKEW_CALIB CSI2_EVT_SHORT_PACKET CSI2_EVT_BIT_NOT_TOGGLE CSI2_EVT_NEXT_START_NOT_0

4.3.2.5 expectedCRC

```
uint16 expectedCRC[CSI2_MAX_VC]
```

Computed CRC for received data, if CSI2_ERR_PACK_CRC reported

4.3.2.6 invalidPacketID

```
uint8 invalidPacketID[CSI2_MAX_VC]
```

The wrong packet ID received, reported if CSI2_ERR_PACK_ID - reported

4.3.2.7 lastDroppedData

```
uint8 lastDroppedData[CSI2_MAX_VC]
```

The last data type dropped on each VC

4.3.2.8 lineLengthErr

```
Csi2_LineLenErrType lineLengthErr[CSI2_MAX_VC]
```

Specific data for line length error, if CSI2_ERR_LINE_LEN or CSI2_ERR_LINE_CNT bits are set

4.3.2.9 notToggledBits

```
uint16 notToggledBits[CSI2_MAX_CHANNEL]
```

Bits mask for not toggled bits, for the channel, if CSI2_EVT_BIT_NOT_TOGGLE reported; the bit mask set to 1 means bit not toggled

4.3.2.10 receivedCRC

```
uint16 receivedCRC[CSI2_MAX_VC]
```

Real, received CRC if CSI2_ERR_PACK_CRC reported

4.3.2.11 shortPackets

```
Csi2_ShortPacketType shortPackets[CSI2_MAX_VC]
```

Specific data for short packet received, if CSI2_EVT_SHORT_PACKET bit is set.

4.3.2.12 unitId

```
uint8 unitId
```

Unit id reporting the error

4.4 Csi2_LineLenErrType Struct Reference

Structure for line length error or number of lines received error in the received frame.

```
#include <Csi2_Types.h>
```

Data Fields

- uint32 linePoz
- uint32 lineLength

4.4.1 Detailed Description

Structure for line length error or number of lines received error in the received frame.

Usually this data is used in an error callback, related to CSI2_EVENTS_IRQ_ID interrupt, in Csi2ErrorReportType↩ ::lineLengthErr.

4.4.2 Field Documentation

4.4.2.1 lineLength

`uint32 lineLength`

The received length (different from the expected one), valid only if line length error is signaled

4.4.2.2 linePoz

`uint32 linePoz`

The first line with length problem (if line/chirp length error) or the total number of lines received (if number of lines error)

4.5 Csi2_MetaDataParamsType Struct Reference

Structure for VirtualChannel configuration of metadata.

`#include <Csi2_Types.h>`

Data Fields

- uint16 streamDataType
- uint16 expectedNumBytes
- uint16 expectedNumLines
- uint16 bufNumLines
- uint16 bufLineLen
- void * bufDataPtr

4.5.1 Detailed Description

Structure for VirtualChannel configuration of metadata.

The structure hold all necessary parameters to initialize the reception of other kind of data than radar data. The main purpose of this structure is to transmit the necessary information to manage the MetaData sent by the Radar Front-End after the data frame was sent.

Note

There is possible to use only one metadata flow for each front-end Virtual Channel. The data is written into the buffer in the received order.

4.5.2 Field Documentation

4.5.2.1 bufDataPtr

```
void* bufDataPtr
```

Pointer to data buffer (16 bytes aligned), physical address

4.5.2.2 bufLineLen

```
uint16 bufLineLen
```

The available line length in buffer, per line (16 bytes aligned) must include 80 supplementary bytes for line/chirp statistics

4.5.2.3 bufNumLines

```
uint16 bufNumLines
```

Available number of complete length lines/chirps (including chirp statistics to be received in buffer, at least 1

4.5.2.4 expectedNumBytes

```
uint16 expectedNumBytes
```

Expected line length, in bytes, for receiving data

4.5.2.5 expectedNumLines

```
uint16 expectedNumLines
```

Expected number of lines/chirps per frame

4.5.2.6 streamDataType

```
uint16 streamDataType
```

Data type to be received

4.6 Csi2_SetupParamsType Struct Reference

Structure for unit configuration.

```
#include <Csi2_Types.h>
```

Data Fields

- uint8 numLanesRx
- uint8 lanesMapRx [CSI2_MAX_LANE]
- uint8 initOptions
- uint8 hfDataManagement
- Csi2_PinsSwapType pinsSwapMask
- uint8 statManagement
- uint32 rxClkFreq
- Csi2_VCParamsType * vcConfigPtr [CSI2_MAX_VC]
- Csi2_VCParamsType * auxConfigPtr [CSI2_MAX_VC]
- Csi2_MetaDataParamsType * metaDataPtr [CSI2_MAX_VC]
- Csi2_IsrCbType pCallback [MIPICSI2_1_INT2_IRQn - MIPICSI2_1_INT0_IRQn+1u]
- rsdkCoreId_t irqExecCore
- uint8_t irqPriority

4.6.1 Detailed Description

Structure for unit configuration.

The structure accepts setup of all available Virtual Channels (VC), but using only one data type for each VC.

4.6.2 Field Documentation

4.6.2.1 auxConfigPtr

```
Csi2_VCParamsType* auxConfigPtr[CSI2_MAX_VC]
```

pointers to VC configurations, for auxiliary data only; The corresponding VC must have set CSI2_VC_BUF_↩ FIFTH_CH_ON; if this pointer is NULL, the the auxiliary data will be dropped. If this pointer is not NULL, and the corresponding VC has CSI2_VC_BUF_FIFTH_CH_ON, it is assumed auxiliary data type 0; so, for auxiliary data type 1 the buffer must be configured, even the data is not used.

4.6.2.2 hfDataManagement

```
uint8 hfDataManagement
```

SAF85XX/SAF86XX specific header/footer reception parameter. This parameter is set at unit level, so all data flows must receive it from FrontEnd.

4.6.2.3 initOptions

```
uint8 initOptions
```

Specific setup options for the unit, required by the application :

- simple/normal calibration or quick/short calibration
- wait for 5us or not wait for STOP states on data lanes use Csi2_DphySetupOptionsType to set this field.

4.6.2.4 irqExecCore

```
rsdkCoreId_t irqExecCore
```

Processor core to execute the irq code. Usually the current core.

4.6.2.5 irqPriority

```
uint8_t irqPriority
```

Priority for the interrupt request execution

4.6.2.6 lanesMapRx

```
uint8 lanesMapRx[CSI2_MAX_LANE]
```

Lanes mapping :

- first byte - the physical lane to be used as lane 1,
- second byte - the physical lane to be used as lane 2,
- etc. use Csi2_LaneIdType to set these fields

4.6.2.7 metaDataPtr

```
Csi2_MetaDataParamsType* metaDataPtr[CSI2_MAX_VC]
```

pointers to VC configurations for other data to be received on the other channels, with no statistics support, mainly for MetaData; ignored if NULL. It is not a must to use one of these for normal application, but if used it must be correlated with the Virtual Channel of the appropriate RadarData.

4.6.2.8 numLanesRx

```
uint8 numLanesRx
```

Number of lanes used to receive data; use Csi2_LaneIdType to configure this.

4.6.2.9 pCallback

```
Csi2_IsrCbType pCallback[MIPICSI2_1_INT2_IRQn - MIPICSI2_1_INT0_IRQn+1u]
```

- the necessary callbacks :

errors in receive path	first callback
errors in protocol & packet level	second callback
events	third callback

At least first callback pointer must be not NULL. When an error is signaled, the appropriate callback is used to signal the occurrence and the associated data. So, any other necessary callback will use the first pointer if NULL specified for that interrupt position.
For events : only when a requested event signal appeared, the **events** callback is used, if specified; if not, **the first** callback will be used.

4.6.2.10 pinsSwapMask

```
Csi2_PinsSwapType pinsSwapMask
```

The necessary polarity change in the CSI2 connector. Default - zero input mask means no polarity change Any other value in bits 0...2 will change the required polarity If the polarity change is not required, the interface will receive no data

4.6.2.11 rxClkFreq

```
uint32 rxClkFreq
```

Receiving (Rx) frequency (between CSI2_MIN_RX_FREQ and CSI2_MAX_RX_FREQ), in Mbps.

4.6.2.12 statManagement

```
uint8 statManagement
```

This field is not available if CSI2_STATISTIC_DATA_USAGE is set to STD_OFF. Use Csi2_AutoDCComputeTimeType to set this field and read carefully how to use the specified values.

4.6.2.13 vcConfigPtr

```
Csi2_VCParamsType* vcConfigPtr[CSI2_MAX_VC]
```

pointers to VC configurations; ignored if NULL. At least VC 0 must be configured.

4.7 Csi2_ShortPacketType Struct Reference

Structure for short packet received.

```
#include <Csi2_Types.h>
```

Data Fields

- uint8 dataID
- uint16 dataVal

4.7.1 Detailed Description

Structure for short packet received.

Usually this data is used in an error callback, related to CSI2_PATH_ERR_IRQ_ID interrupt, in Csi2_ErrorReportType::shortPackets.

4.7.2 Field Documentation

4.7.2.1 dataID

```
uint8 dataID
```

Identifier (ID) of the packet

4.7.2.2 dataVal

```
uint16 dataVal
```

Data value (16 bits)

4.8 Csi2_VCDriverStateType Struct Reference

Structure for VC driver parameters.

`#include <Csi2_Types.h>`

Data Fields

- uint8 eventsMask
- uint16 outputDataMode
- uint16 lastReceivedBufLine
- uint16 lastReceivedChirpLine
- Csi2_VCParamsType * vcParamsPtr
- Csi2_ChFrameStatType statDC [CSI2_MAX_CHANNEL]
- uint8 metaDataUsage

4.8.1 Detailed Description

Structure for VC driver parameters.

Internal parameters for easy driver usage.

4.8.2 Field Documentation

4.8.2.1 eventsMask

`uint8 eventsMask`

4.8.2.2 lastReceivedBufLine

`uint16 lastReceivedBufLine`

4.8.2.3 lastReceivedChirpLine

`uint16 lastReceivedChirpLine`

4.8.2.4 metaDataUsage

```
uint8 metaDataUsage
```

4.8.2.5 outputDataMode

```
uint16 outputDataMode
```

4.8.2.6 statDC

```
Csi2_ChFrameStatType statDC[CSI2_MAX_CHANNEL]
```

4.8.2.7 vcParamsPtr

```
Csi2_VCParamsType* vcParamsPtr
```

4.9 Csi2_VCParamsType Struct Reference

Structure for VirtualChannel configuration.

```
#include <Csi2_Types.h>
```

Data Fields

- uint16 streamDataType
- uint8 channelsNum
- uint8 vcEventsReq
- uint16 expectedNumSamples
- uint16 expectedNumLines
- Csi2_BufferPointerResetType bufferReset
- uint16 bufNumLines
- uint16 bufLineLen
- uint16 outputDataMode
- void * bufDataPtr
- sint16 offsetCompReal [CSI2_MAX_CHANNEL]
- sint16 offsetCompImg [CSI2_MAX_CHANNEL]
- uint8 gpio1Mask
- uint8 gpio2Mask
- uint8 gpio1EnaMask
- uint8 gpio2EnaMask
- uint8 sdma1Mask
- uint8 sdma2Mask
- uint8 sdma1EnaMask
- uint8 sdma2EnaMask
- uint8 bufNumLinesTrigger

4.9.1 Detailed Description

Structure for VirtualChannel configuration.

The structure hold all necessary parameters to setup the reception of the data for the Virtual Channel (VC).

Note

Assuming any VC is receiving only one data_type, so using only one buffer.

4.9.2 Field Documentation

4.9.2.1 bufDataPtr

```
void* bufDataPtr
```

Pointer to data buffer (16 bytes aligned), physical address

4.9.2.2 bufferReset

```
Csi2_BufferPointerResetType bufferReset
```

The way data is written into buffer after Frame Start

4.9.2.3 bufLineLen

```
uint16 bufLineLen
```

The available line length in buffer, per line (16 bytes aligned) must include 80 supplementary bytes for line/chirp statistics

4.9.2.4 bufNumLines

```
uint16 bufNumLines
```

Available number of complete length lines/chirps (including chirp statistics to be received in buffer, at least 1

Note

For internal software reasons, (expectedNumLines % bufNumLines) must not be 1

4.9.2.5 bufNumLinesTrigger

```
uint8 bufNumLinesTrigger
```

Number of lines received in buffer to generate a callback to application, if the CSI2_EVT_LINE_END is required in vcEventsReq. It correspond to RM - Table 81-7 - LINEDONE field and cap. 81.8.41

4.9.2.6 channelsNum

```
uint8 channelsNum
```

Channels number for this VC; equal to antennas number for radar

4.9.2.7 expectedNumLines

```
uint16 expectedNumLines
```

Expected number of lines/chirps per frame

4.9.2.8 expectedNumSamples

```
uint16 expectedNumSamples
```

Expected line length for receiving in terms of samples/pixels/etc. per channel (antenna for radar)

4.9.2.9 gpio1EnaMask

```
uint8 gpio1EnaMask
```

4.9.2.10 gpio1Mask

```
uint8 gpio1Mask
```

4.9.2.11 gpio2EnaMask

```
uint8 gpio2EnaMask
```

Mask for GPIO triggers enablement (see CSI2_REQ_ETRG_ENA_... family, i.e. CSI2_REQ_ETRG_ENA_ON_↩ ERROR); must be set correctly to have external trigger of the related events.

4.9.2.12 gpio2Mask

```
uint8 gpio2Mask
```

masks for GPIO trigger handling (see CSI2_REQ_ETRG_EVT_... family, i.e. CSI2_REQ_ETRG_EVT_FRAME_↩ START); to be functional, the triggers must be enabled (see gpio1EnaMask and gpio2EnaMask

4.9.2.13 offsetCompImg

```
sint16 offsetCompImg[CSI2_MAX_CHANNEL]
```

Channel offset compensation for imaginary part, see offsetCompReal

4.9.2.14 offsetCompReal

```
sint16 offsetCompReal[CSI2_MAX_CHANNEL]
```

Channel offset compensation for real part,
CSI2_OFFSET_AUTOCOMPUTE => auto config_DC,
any other = compensation (0 => no compensation).
The compensation must be specified at AD input level (12 bits).

4.9.2.15 outputDataMode

```
uint16 outputDataMode
```

The mode data is output in the buffer and other information about input data, see above : CSI2_VC_BUF_↩ ... family definitions i.e. CSI2_VC_BUF_COMPLEX_DATA. This parameter must be identical for data buffer and for correspondent auxiliary buffer.

4.9.2.16 sdma1EnaMask

```
uint8 sdma1EnaMask
```

4.9.2.17 sdma1Mask

```
uint8 sdma1Mask
```

4.9.2.18 sdma2EnaMask

`uint8 sdma2EnaMask`

masks for SDMA trigger handling (see CSI2_REQ_ETRG_EVT_... family, i.e. CSI2_REQ_ETRG_EVT_FRAME↩ _START); to be functional, the triggers must be enabled (see sdma1EnaMask and sdma2EnaMask)

4.9.2.19 sdma2Mask

`uint8 sdma2Mask`

masks for SDMA trigger handling (see CSI2_REQ_ETRG_EVT_... family, i.e. CSI2_REQ_ETRG_EVT_FRAME↩ _START); to be functional, the triggers must be enabled (see sdma1EnaMask and sdma2EnaMask)

4.9.2.20 streamDataType

`uint16 streamDataType`

Data type to be received

4.9.2.21 vcEventsReq

`uint8 vcEventsReq`

Mask for IRQ requested events for VC (i.e. CSI2_EVT_FRAME_END or similar)

Index

How to Reach Us:

Home Page:
nxp.com

Web Support:
nxp.com/support